

OS Practical 12

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 20

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg) {
    for (int i = 1; i <= ITEMS; i++) {

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Produced: %d\n", i);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        usleep(100000);
    }

    return NULL;
}

void *consumer(void *arg) {
    for (int i = 1; i <= ITEMS; i++) {

        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;
    }
}

```

```

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        usleep(150000);
    }

    return NULL;
}

int main() {
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}

```

```
nishanth@LAPTOP-6Q3ON07H:~/q12$ ./producer_consumer
```

```
Produced: 1
```

```
Consumed: 1
```

```
Produced: 2
```

```
Consumed: 2
```

```
Produced: 3
```

```
Consumed: 3
```

```
Produced: 4
```

```
Produced: 5
```

```
Consumed: 4
```

```
Produced: 6
```

```
Consumed: 5
```

```
Produced: 7
```

```
Produced: 8
```

```
Consumed: 6
```

```
Produced: 9
```

```
Consumed: 7
```

```
Produced: 10
```

```
Produced: 11
```

```
Consumed: 8
```

```
Produced: 12
```

```
Consumed: 9
```

```
Produced: 13
```

```
Produced: 14
```

```
Consumed: 10
```

```
Produced: 15
```

```
Consumed: 11
```

```
Produced: 16
```

```
Consumed: 12
```

```
Produced: 17
```

```
Consumed: 13
```

```
Produced: 18
```

```
Consumed: 14
```

```
Produced: 19
```

```
Consumed: 15
```

```
Produced: 20
```

```
Consumed: 16
```

```
Consumed: 17
```

```
Consumed: 18
```

```
Consumed: 19
```

```
Consumed: 20
```

Try:

```
#define BUFFER_SIZE 1
```

```
nishanth@LAPTOP-6Q3ON07H:~/q12$ nano producer_consumer.c
nishanth@LAPTOP-6Q3ON07H:~/q12$ gcc producer_consumer.c -o producer_consumer -pthread
nishanth@LAPTOP-6Q3ON07H:~/q12$ nano producer_consumer.c
nishanth@LAPTOP-6Q3ON07H:~/q12$ gcc producer_consumer.c -o producer_consumer -pthread
nishanth@LAPTOP-6Q3ON07H:~/q12$ time ./producer_consumer
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
Produced: 6
Consumed: 6
Produced: 7
Consumed: 7
Produced: 8
Consumed: 8
Produced: 9
Consumed: 9
Produced: 10
Consumed: 10
Produced: 11
Consumed: 11
Produced: 12
Consumed: 12
Produced: 13
Consumed: 13
Produced: 14
Consumed: 14
Produced: 15
Consumed: 15
Produced: 16
Consumed: 16
Produced: 17
Consumed: 17
Produced: 18
Consumed: 18
Produced: 19
Consumed: 19
Produced: 20
Consumed: 20

real    0m3.009s
user    0m0.001s
sys     0m0.009s
```

nano deadlock.c:

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg) {

    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg) {

    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    sleep(1);

    printf("Thread 2: waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: acquired both resources\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main() {

    pthread_t t1, t2;

```

```
nishanth@LAPTOP-6Q30N07H:~/q12$ nano deadlock.c
nishanth@LAPTOP-6Q30N07H:~/q12$ gcc deadlock.c -o deadlock -pthread
nishanth@LAPTOP-6Q30N07H:~/q12$ ./deadlock
Thread 1: locking Resource 1
Thread 2: locking Resource 2
Thread 1: waiting for Resource 2
Thread 2: waiting for Resource 1
^C
```

Then the program will appear frozen.

That is the deadlock.

To stop it:

Ctrl + C

Prevent deadlock using resource ordering:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg) {

    printf("Thread 1: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg) {

    printf("Thread 2: locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 2: locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main() {

    pthread_t t1, t2;
```

```

pthread_t t1, t2;

pthread_mutex_init(&resource1, NULL);
pthread_mutex_init(&resource2, NULL);

pthread_create(&t1, NULL, thread1, NULL);
pthread_create(&t2, NULL, thread2, NULL);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

pthread_mutex_destroy(&resource1);
pthread_mutex_destroy(&resource2);

return 0;
}

```

```

nishanth@LAPTOP-6Q3ON07H:~/q12$ nano deadlock_prevention.c
nishanth@LAPTOP-6Q3ON07H:~/q12$ gcc deadlock_prevention.c -o deadlock_prevention -pthread
nishanth@LAPTOP-6Q3ON07H:~/q12$ ./deadlock_prevention
Thread 1: locking Resource 1
Thread 2: locking Resource 1
Thread 1: locking Resource 2
Thread 1: acquired both resources
Thread 2: locking Resource 2
Thread 2: acquired both resources
nishanth@LAPTOP-6Q3ON07H:~/q12$

```